

ENTERPRISE ACCESS FOUNDATION

Phase 1.5 Target Architecture & Execution Boundary

Authorization / Access / Approval / Policy / Territory / Field Security

Enterprise Super App - Architecture Decision Report

Foundation-first, not MVP-first

1. Executive Decision

Phase 1.5 should not remain a simple Role -> Permission baseline. It should be redefined as the Enterprise Access Foundation: a stable authorization control-plane foundation that can later accept organization scopes, record sharing, territory facts, field security, ABAC restrictions, approvals and automation without replacing the core model.

AUTHORIZATION != ORGANIZATION != TERRITORY != SHARING
!= FIELD SECURITY != APPROVAL != AUTOMATION

These capabilities may share primitives such as PrincipalRef, ResourceRef, ActionKey, ScopeRef, ConditionExpression, EffectivePeriod, DecisionId and ConfigurationRevision, but they should not be collapsed into one generic Rules table or one universal rule engine.

2. Current State

- CRM Phase 0.5 MasterData/Party foundation is complete.
- CRM Phase 1 Lifecycle/Pipeline foundation is complete.
- Phase 2 command/API work is intentionally gated by Phase 1.5 Access.
- CRM already has a strong tenant-safety spine: PostgreSQL RLS, tenant-safe relationships, evidence, idempotency, concurrency and outbox patterns.
- Authorization enforcement is not yet wired in front of CRM commands.
- The repo-derived analysis describes the current Access module as existing but not yet at the same RLS/test/Host-wiring maturity as CRM/MasterData.

This is the right moment to redesign the Access foundation. Once Phase 2 commands, APIs, search, exports, extensions and approval flows depend on the old permission semantics, migration cost rises sharply.

3. Leader Benchmark - Common Industry Pattern

Platform	Functional authority	Record/data scope	Field	Org/Territory	Approval
Salesforce	Profiles / Permission Sets / Permission Set Groups	OWD + ownership + role hierarchy + sharing	FLS	Territory model separate	Approval process separate
Dynamics 365 / Dataverse	Security Roles + privileges	User / BU / Parent-Child BU / Organization + sharing	Column security	Business Unit hierarchy	Approvals/ workflows separate
SAP	Business Role + Business Catalog	Authorization restrictions / org values	Authorization restrictions	Sales org / company / plant etc.	Flexible Workflow separate
Oracle Fusion	Job Role -> Duty Role -> Privilege	Data Security Policies / Data Roles	Data security	Org/security profiles	Approval routing separate
ServiceNow	Roles + ACL	Record/table ACL conditions	Field ACL	Group/org facts	Flow/approval separate
HubSpot	Permissions / permission sets	Own / Team / All	Property restrictions	Teams	Approvals separate
Zoho	Profiles / roles	Hierarchy + sharing + territories	Module/field controls	Role/territory hierarchy	Approval process separate

Cross-vendor conclusion: serious enterprise platforms separate what the user may do from which data/records they may do it on. Approval and territory are not treated as mere role flags.

4. Final Authorization Algebra

```
Effective Authorization =  
  Tenant Boundary  
  ∩ Functional Grant  
  ∩ Record / Data Scope  
  ∩ Relationship / Sharing  
  ∩ Restriction Policies  
  ∩ Field Security
```

Normal role/permission access should use default-deny plus additive grants. A separate restriction layer may narrow or forbid access under security conditions. General DENY entries should not be mixed into ordinary Role/PermissionSet inheritance.

5. Core Functional Model

```
Role  
  |  
  v  
PermissionSet  
  |  
  v  
Permission / ActionKey
```

Permission is the smallest stable business capability. PermissionSet is a reusable business-duty/capability bundle. Role is a tenant-facing security persona. Organization hierarchy must not be modeled as role inheritance.

Concept	Meaning	Example
Permission / ActionKey	Atomic business action	crm.opportunity.win
PermissionSet	Reusable business duty bundle	CRM Opportunity Management
Role	Security persona	Sales Manager
RoleAssignment	Principal receives role in a scope/time window	Deniz -> Sales Manager
PermissionSetAssignment	Exceptional/direct capability bundle assignment	Ayşe -> Quote Approver until 2026-10-01

6. Direct Assignment Policy

Preferred normal path:

```
Principal -> Role -> PermissionSet -> Permission
```

Allowed exceptional path:

```
Principal -> PermissionSet -> Permission
```

Direct Principal -> Permission grants should not be the standard model. Temporary/JIT/delegated access can be represented through effective-dated PermissionSet assignments without creating ad-hoc roles or opaque user grants.

7. Scope and Relationship Semantics

Concept	Classification	Target semantic
Tenant / All	Scope	Tenant-wide
Organization	Hierarchical scope	OrgNodeRef + HierarchyRef + traversal
Territory	Hierarchical/business scope	TerritoryRef + TerritoryModelVersionRef

Own	Relationship	resource.owner == principal
Team	Relationship	principal memberOf team AND resource owned/assigned to team
Sharing	Explicit/additive record grant	resource sharedWith user/team/group

A key freeze decision is that Own/Team/Sharing and Organization/Territory are not the same data type. The former are relationships/grants; the latter are organizational scopes/hierarchies.

8. Record Access Rules

```

Functional Permission
+
Grant paths:
- Tenant scope
- Ownership
- Team relationship
- Organization scope
- Territory relationship
- Explicit sharing
|
v
Restriction policies
|
v
Field restrictions

```

A record share must not create a business capability that the principal does not already possess. Sharing can widen record visibility only within the ceiling established by functional permissions.

9. Domain Ownership

Domain	Owns	Relationship
Access / Authorization	Action registry, PermissionSets, Roles, assignments, PDP/PEP, decision reasoning, configuration audit	Consumes org/territory/sharing/field facts
Organization Structure	Legal entities, BUs, departments, branches, teams, positions, memberships, management hierarchies	Fact provider to Access/Approval
Territory	Territory models, versions, hierarchy, memberships, resource assignments/rules	Fact provider to Access/Approval
Sharing	Explicit record grants + provenance	Grant provider to Access
Field Security	Sensitive field policy/evaluator	Restriction layer
Approval	Definitions/versions, steps, approver resolution, instances, decisions	Consumes Access + org/territory facts
Automation / Rules	Trigger / Condition / Action definitions and runs	Executes authorized domain commands

10. Territory Model

Territory must be a separate bounded context, not a Role scope enum. The future model should support versioned territory structures and resource/principal assignments.

```

TerritoryModel
-> TerritoryModelVersion
-> Territory hierarchy
-> TerritoryMembership

```

```
-> ResourceTerritoryAssignment
-> TerritoryAssignmentRule
```

Phase 1.5 should reserve the evaluator/provider contract but should not create the territory engine or empty territory tables.

11. Organization Structure

Access must not own the organization tree. It should consume organization facts through contracts. Security must also avoid assuming a single parent hierarchy forever; management, commercial and reporting hierarchies may differ.

12. Sharing Model

```
RecordShare
- TenantId
- ResourceRef
- GranteeRef
- Rights
- Source
- SourceRef
- EffectiveFrom / EffectiveTo
- CreatedBy / CreatedAt
```

Suggested Source values: Manual, Rule, Team, Territory, Process, System.

Provenance is mandatory so the platform can answer 'why can this principal see this record?' without guesswork.

13. Field Security

Field security is a separate evaluator and should be enforced across API serialization, reads, writes, exports, search, extensions and AI context - not only via frontend hiding.

```
Example FieldAccessSet
estimated_amount -> Read/Write
margin           -> Read
cost             -> Hidden
phone           -> ReadMasked
```

Do not turn every ordinary field into an independent permission. Only security-relevant fields need explicit security metadata/policy.

14. Approval Domain

Approval is not Authorization. Authorization answers whether a principal is permitted to perform an action; approval defines whether a business transaction requires human/process consent, which steps apply and who resolves those steps.

```
ApprovalDefinition
-> ApprovalDefinitionVersion
-> ApprovalStep
-> StepCondition
-> ApproverResolverSpec
-> ApprovalInstance
-> ApprovalStepInstance
```

```
-> ApprovalTask
-> ApprovalDecision
```

ApprovalInstance must be pinned to an immutable ApprovalDefinitionVersion. New definitions apply to new instances; in-progress approvals must not silently remap.

14.1 Approver Resolution

- SpecificPrincipal
- Team
- ManagerOf
- ManagementHierarchy
- PositionHierarchy
- SecurityRole
- OrganizationResponsibility
- TerritoryResponsibility
- CustomResolver

Being selected as an approver should not automatically grant unrestricted access to the underlying business record. Approval task visibility and business-object permissions should be separate, with the approval runtime exposing only the snapshot/evidence necessary to decide when normal record access is absent.

15. Dynamic Approval Conditions

```
IF quote.discount > 20%
THEN Regional Sales Manager
```

```
IF quote.amount > 500000
THEN CFO
```

Condition-expression infrastructure may eventually be shared as a library, but approval conditions remain owned by the Approval domain. They must not be stored as generic Access policies.

16. Automation / Rules

```
AutomationDefinition
-> AutomationDefinitionVersion
    -> Trigger
    -> Condition
    -> Action
-> AutomationRun
```

Automation actions must execute as an explicit machine/service principal through the same authorization and domain-command contracts as human actions. Automation must never bypass Access because it is 'system generated'.

17. Why There Is No Generic Rule Engine

Domain	Input	Output
Authorization	Principal + Action + Resource + Context	Permit / Deny
Approval	Transaction facts + conditions	Approval steps / approvers
Territory	Resource/principal facts + assignment criteria	Territory assignment
Automation	Trigger + conditions	Actions
Sharing	Record criteria / explicit grant	Record visibility grant

A reusable ConditionExpression/AST may be shared later, but rule lifecycle, ownership, versioning and evaluation semantics must remain domain-specific. Avoid a generic rules(type, trigger_json, condition_json, action_json) table.

18. ABAC and ReBAC Readiness

The canonical authorization request should be stable from Phase 1.5:

```
Authorize(  
  Principal,  
  Action,  
  Resource,  
  Context  
) -> AuthorizationDecision
```

The first evaluator can implement RBAC + TenantScope only. Future ABAC may add subject/resource/environment attributes; future ReBAC may add owner/member/sharedWith/managerOf relations without breaking the command contract.

19. Query Authorization Is a First-Class Contract

Single-resource checks and list/search authorization are different problems. Enterprise list screens must not call Authorize() remotely once per row.

```
Authorize(request)          -> one resource decision  
ResolveAccessScope(request) -> query/filter scope
```

Phase 1.5 may initially resolve only TenantScope, but the contract must have room for ownership, organization, territory and sharing filters later.

20. Authorization Evaluation Pipeline

```
REQUEST  
-> Trusted Actor Context  
-> Tenant Boundary / RLS  
-> Action Registry Validation  
-> Functional Grants (Role / PermissionSet)  
-> Grant Paths (Tenant / Own / Team / Org / Territory / Sharing)  
-> Restriction Layer (ABAC / sensitivity / temporal / security context)  
-> Field Security  
-> ALLOW / DENY  
-> Domain Business Policy  
-> Approval Requirement / Approval State  
-> Domain Invariants  
-> Command Execution
```

21. RLS vs Application Authorization

Layer	Responsibility
PostgreSQL RLS	Hard tenant isolation / defense in depth
Access Control Plane	Within-tenant business/resource authorization
Domain policies	Lifecycle/state/business invariants
Approval	Required human/process consent

Organization/territory/ownership/sharing semantics should not be embedded as the primary source of truth in PostgreSQL RLS, because the same authorization must govern APIs, background workers, exports, search, extensions and AI.

22. Cross-Module Authorization Contract

```
CRM -----Sales ----- Finance ---- > Contracts <---- Access implementation
Inventory - /
HR -----/
```

Business modules must depend on a stable authorization contract, not on Access.Persistence or an Access-specific ORM model.

```
AuthorizationDecision
- Effect
- ReasonCode
- DecisionId
- ConfigurationRevision
- MatchedGrantRefs
- MatchedRestrictionRefs
- Obligations
```

23. Action Registry Ownership

The owning domain defines the business action vocabulary; Access evaluates it. CRM owns `crm.opportunity.*`, Sales owns `sales.quote.*`, etc. The registry should carry metadata such as owner module, resource type, risk class and lifecycle status. CI should reject unregistered ActionKeys.

24. Audit, Evidence and Versioning

Concern	Target
Access configuration audit	Who changed roles/sets/assignments; before/after; revision
Authorization decision telemetry	Actor, action, resource, result, reason, revision, correlation
High-risk decision evidence	Durable linkage to AuthorizationDecisionId
Business evidence	Owned by CRM/Sales/etc.; not replaced by access audit
Approval versioning	Instance pinned to immutable definition version
Access revision	Monotonic tenant configuration revision carried in decisions

Do not synchronously insert every normal read decision into a relational table. Configuration changes and high-risk mutations need durable evidence; normal reads can use structured decision telemetry/logging with sampling/retention rules.

25. Tenant Configurability Boundary

Platform-owned	Tenant-configurable
Resource types / ActionKeys / semantics	Roles
Evaluator contracts / security invariants	PermissionSet composition
Action registry identifiers	Assignments and effective periods
Authorization request/decision contract	Scope bindings
Cross-module security contracts	Later: sharing, field security, territories, approvals, automations

26. Phase 1.5 - Real Implementation Scope

Capability	Phase 1.5 decision
Action / Permission Registry	IMPLEMENT NOW
Permission	IMPLEMENT NOW
PermissionSet	IMPLEMENT NOW
Role	IMPLEMENT NOW
Role -> PermissionSet	IMPLEMENT NOW
Principal -> Role Assignment	IMPLEMENT NOW
Direct PermissionSet Assignment	IMPLEMENT NOW
Effective-dated assignments	IMPLEMENT NOW
Tenant / All scope	IMPLEMENT NOW
Authorize() contract / PDP	IMPLEMENT NOW
Server-side enforcement seam / PEP	IMPLEMENT NOW
Query-scope contract	IMPLEMENT NOW
Authorized-actions discovery	IMPLEMENT NOW
Default deny	IMPLEMENT NOW
Configuration revision + audit	IMPLEMENT NOW
Decision reasoning/telemetry	IMPLEMENT NOW
Access RLS + negative tests	IMPLEMENT NOW
Action registry consistency tests	IMPLEMENT NOW
Org/Team/Ownership resolver seams	DESIGN/FREEZE ONLY
Sharing	DESIGN/FREEZE ONLY
Territory	DESIGN/FREEZE ONLY
Field Security	DESIGN/FREEZE ONLY
ABAC restrictions	DESIGN/FREEZE ONLY
Approval / Approver resolution	DESIGN/FREEZE ONLY
Automation / Rules	DESIGN/FREEZE ONLY
SoD / JIT / Delegation	DESIGN/FREEZE ONLY

27. Do Not Build in Phase 1.5

- Generic all-purpose rule engine
- Full ABAC DSL / arbitrary scripting
- OpenFGA / OPA / Cedar runtime dependency
- Territory engine
- Approval runtime / workflow designer
- Field-security administration UI
- Sharing administration UI
- SoD engine
- Full privileged-access/JIT system

Foundation-first is not speculative-database-first. Reserve interfaces, semantics and stable identifiers now; create full persistence and runtime only when a concrete phase consumes the capability.

28. Freeze Decisions

Decision	Status
Tenant isolation != authorization	FREEZE
Default deny	FREEZE
Functional grants are additive	FREEZE
Restriction layer is separate from grants	FREEZE
Permission = stable ActionKey	FREEZE
PermissionSet is first-class	FREEZE
Role = security persona	FREEZE
Role != organization hierarchy	FREEZE

No normal direct Permission grants	FREEZE
Direct PermissionSet assignment allowed	FREEZE
Assignments are effective-dated	FREEZE
Own/Team/Sharing = relationship/grant	FREEZE
Org/Territory = organizational scope	FREEZE
Sharing cannot exceed functional capability	FREEZE
Territory separate bounded context	FREEZE
Organization separate bounded context	FREEZE
Approval separate bounded context	FREEZE
Automation separate bounded context	FREEZE
Field Security separate evaluator	FREEZE
Principal + Action + Resource + Context is canonical request	FREEZE
Query authorization is separate contract	FREEZE
Config revision is returned with decisions	FREEZE
Approval instance is pinned to definition version	FREEZE
RLS initially tenant-only	FREEZE
Domain owns ActionKeys	FREEZE
Access implementation hidden behind Contracts	FREEZE
No generic all-purpose Rules table	FREEZE

29. Migration and YAGNI Risk

Migration risk is currently favorable because the Access foundation has not yet become the enforcement dependency for the next CRM commands/API surface. The major risk is delaying this redesign until Phase 2/3, when handlers, HTTP APIs, search, exports, extensions and approvals may already depend on the old semantics.

- Reuse existing Role/Permission/TenantId/PrincipalRef primitives when their semantics match.
- Do not preserve Role -> Permission as the public contract merely because it already exists.
- Prefer additive migration toward PermissionSet + Assignment + Scope semantics.
- Avoid creating future-domain tables until a concrete capability consumes them.
- Keep engine choice (internal evaluator vs Cedar/OPA/OpenFGA-like runtime later) behind contracts.

30. Final Target Architecture

```

IDENTITY
|
Trusted ActorContext
|
ACCESS FOUNDATION
- Action Registry
- Permission
- PermissionSet
- Role
- Assignment
- Scope
- PDP / PEP
- Query Scope
- Decision Audit
|
+-- Organization facts
+-- Sharing facts
+-- Territory facts
+-- Field Security
+-- ABAC restrictions
|
Authorization Decision
|
Domain Business Policy

```

```
|
Approval Runtime (when required)
|
Domain Command / Evidence

Automation
Trigger -> Condition -> Action
|
Authorized Domain Command
```

31. Final Recommendation

Decision: COMPLETELY REVISE the old Phase 1.5 plan.

The old Role -> Permission baseline is too narrow as an architecture boundary. Existing compatible primitives may be reused, but Phase 1.5 should be rewritten as an Enterprise Access Foundation plan. The implementation should stop at the core access control plane and stable extension seams; territory, sharing, field security, ABAC, approval and automation should be designed/frozen but not implemented yet.

32. Reference Sources Used for Benchmarking

- Salesforce Help / Trailhead - Profiles, Permission Sets, Permission Set Groups, Sharing Model, Field-Level Security, Enterprise Territory Management, Restriction Rules.
- Microsoft Learn - Dataverse security roles and privileges, business units, owner/access teams, column-level security.
- Microsoft Learn - Dynamics 365 Finance & Operations role-based security and Extensible Data Security (XDS).
- SAP Help - S/4HANA Cloud business roles/catalogs/authorization restrictions; Flexible Workflow; Responsibility Management.
- Oracle Cloud Docs - Job/Duty/Privilege security, Data Security Policies, approval rules and approver types.
- ServiceNow Docs - ACL evaluation for table/record/field access.
- HubSpot Knowledge Base - User permissions and quote approvals.
- Zoho CRM Help - Profiles, sharing rules, territories and approval process.
- NIST SP 800-162 - Attribute Based Access Control (ABAC).
- Cedar Policy Language - Principal / Action / Resource / Context authorization model and permit/forbid semantics.
- OpenFGA - Relationship-based authorization concepts and immutable authorization models.
- Open Policy Agent - Authorization filtering/search and decision logging concepts.

End of architecture decision report